

Best MVPs for Portfolio 2026

What are best MVPs to develop for my portfolio in 2026?

Given what I know about your background (Next.js, Cloudflare Workers, D1, Node.js, AI, Stripe, and local LLMs), I'd focus on **products that demonstrate engineering depth**, not just CRUD apps.

In 2026, employers and clients are increasingly looking for developers who can build AI-powered, production-ready applications rather than simple dashboards.

1. AI Customer Support Agent (Top Choice)

Build a SaaS where businesses can:

- Upload PDFs, FAQs, and websites
- AI answers customer questions
- Human handoff
- Live chat widget
- Analytics dashboard
- Multi-tenant support

Tech:

- Next.js
- Cloudflare Workers
- D1
- R2
- Qwen/OpenAI
- Vector search
- Stripe

Demonstrates:

- RAG
- AI
- SaaS architecture
- Payments
- Multi-tenancy

2. AI Coding Agent

Like a mini Claude Code.

Features:

- GitHub repository upload
- Repository indexing
- Ask questions about code
- Generate pull requests
- Explain bugs
- File search
- Terminal execution (sandbox)

Very impressive because it demonstrates:

- LLM orchestration
 - Vector search
 - Git integration
 - Streaming
-

3. AI Expense Analyzer

You've already worked in this area.

Expand it into:

- Upload bank statements
- OCR
- Transaction categorization
- Budget recommendations
- Spending forecasts
- Monthly reports
- AI financial advisor

This demonstrates:

- ML
 - NLP
 - Document parsing
 - Analytics
-

4. Shopify AI Assistant

Merchants upload products.

AI can:

- Generate descriptions
- SEO
- Blog posts
- Email campaigns
- Product recommendations
- Customer support

Since Shopify developers are in demand, this is valuable.

5. AI Document Intelligence

Upload:

- PDFs
- DOCX
- Images

Extract:

- Tables
- Key-value pairs
- Summaries
- Structured JSON
- Semantic search

Great showcase of document AI.

6. Developer API Platform

A backend service providing:

- Authentication
- API keys
- Rate limiting
- Billing
- Analytics
- Usage dashboard

- Webhooks
- SDKs

Essentially a small version of Stripe or Resend.

Shows backend engineering skills.

7. Real-Time Taxi Platform

You've explored this architecture before.

Include:

- Driver app
- Rider app
- Live location
- ETA
- Payments
- Admin dashboard
- WebSockets
- Redis
- Notifications

Excellent for demonstrating scalable real-time systems.

8. AI Meeting Assistant

Features:

- Upload recordings
 - Speech-to-text
 - Action items
 - Task extraction
 - Calendar integration
 - Email summaries
-

9. AI Website Builder

User types:

"Create a landing page for a gym."

AI generates:

- React code
- Tailwind
- Images
- Deploy preview
- Editable components

Shows advanced AI workflows.

10. AI Workflow Automation

Similar to Zapier or n8n, but AI-driven.

Example:

"Whenever Stripe payment succeeds, send an invoice and notify Slack."

Includes:

- Visual workflow editor
- AI-generated flows
- Connectors
- Scheduling
- Webhooks

This is a strong systems design project.

Bonus: LLM Gateway

This aligns especially well with your recent interest in local inference.

Features:

- Route requests to different models
- Prompt caching
- User quotas
- Cost tracking
- Streaming
- Automatic model selection
- Retry logic
- Continuous batching
- Metrics dashboard

This showcases AI infrastructure skills.

Infrastructure to Include in Every Project

These are often what differentiate portfolio projects from production-quality work:

- Authentication (JWT or OAuth)
 - Stripe subscriptions
 - Email notifications
 - Logging
 - Monitoring
 - Error tracking
 - CI/CD
 - Background jobs
 - File uploads
 - API documentation
 - Role-based access control
 - Unit and integration tests
 - Docker
 - Rate limiting
 - Security headers
 - Analytics
-

Tech Stack I'd Recommend

- **Frontend:** Next.js + React + Tailwind CSS
 - **Backend:** Cloudflare Workers or Node.js
 - **Database:** D1 or PostgreSQL
 - **ORM:** Drizzle ORM
 - **Storage:** Cloudflare R2
 - **Authentication:** Better Auth or Auth.js
 - **Payments:** Stripe
 - **AI:** Qwen 3.5/3.6, OpenAI, or Gemini
 - **Deployment:** Cloudflare
 - **Realtime:** WebSockets or Durable Objects where appropriate
 - **Monitoring:** OpenTelemetry or Sentry
-

If you only build three projects

I'd choose these because they cover AI, SaaS, backend engineering, and real-time systems:

1. **AI Customer Support SaaS** – demonstrates RAG, multi-tenancy, payments, and production AI.
2. **LLM Gateway & Inference Platform** – highlights AI infrastructure, routing, caching, and local/cloud model orchestration.
3. **Real-Time Taxi Platform** – showcases WebSockets, scalable backend architecture, maps, notifications, and live state synchronization.

Together, those three projects communicate a broad range of modern full-stack and AI engineering skills and make for a compelling 2026 portfolio.

AI Customer Support Agent

Lets break down this problem
what components do I need
for LLM let use NVIDIA: Nemotron 3 Super (free)
<https://openrouter.ai/collections/free-models>

Is RAG all about the context that Model learns?

Great choice. An **AI Customer Support Agent** is one of the strongest portfolio projects because it combines modern AI techniques with production SaaS engineering.

First, to answer your RAG question:

Is RAG all about the context that the model learns?

Not exactly.

The model **doesn't learn or retrain** from your documents. Instead, it receives relevant information as part of the prompt each time it answers.

The flow looks like this:

```
User asks:
"How do I return an order?"

↓

System searches company documents

↓

Finds the relevant paragraphs

↓

Prompt becomes:

System:
You are a helpful support agent.

Context:
"Our return policy allows returns within 30 days..."
```

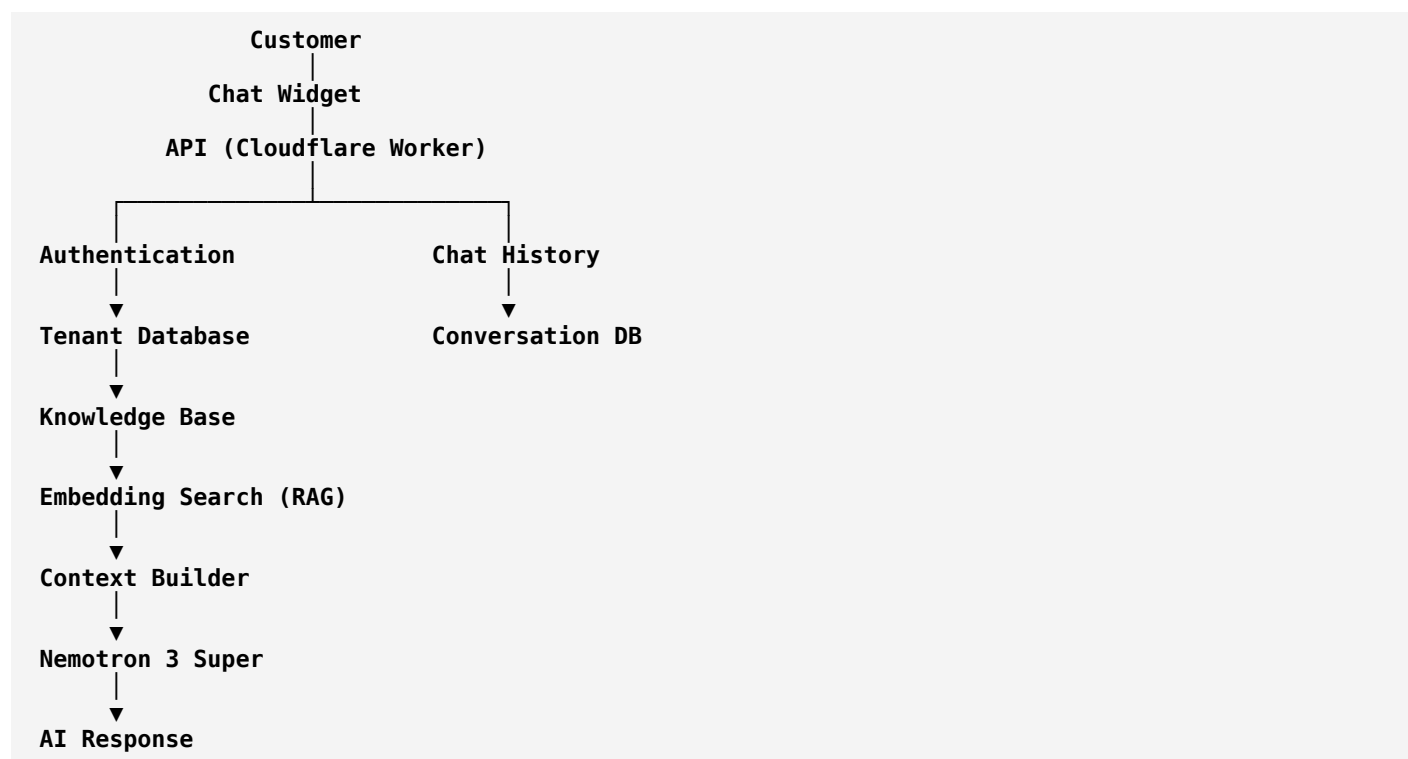
User:
How do I return an order?

↓

LLM generates the answer.

The documents remain in your database. The LLM has **no permanent memory** of them.

Overall Architecture



There are two main areas:

1. **Admin Portal** (what the business uses)
2. **Customer Chat** (what visitors use)

Admin Portal

This is where companies manage their AI assistant.

Authentication

- Sign up
- Login
- Password reset
- Organizations (multi-tenant)

Dashboard

Show things like:

- Total chats
- AI resolution rate
- Human handoffs
- Documents indexed
- Token usage

Knowledge Base

This is the heart of RAG.

Users can upload:

- PDF
- DOCX
- TXT
- Markdown
- FAQs
- Website URLs

Example:

Upload:

Return Policy.pdf

Shipping.pdf

Warranty.pdf

FAQ.docx

Document Processing

When a document is uploaded:

PDF

↓

Extract text

↓

Split into chunks

↓

Generate embeddings

↓

Store in vector database

The original document is also stored (for example, in Cloudflare R2).

Embeddings

Suppose a PDF contains:

Returns

**Customers may return products
within 30 days...**

It becomes chunks like:

Chunk 1

Returns

Customers may return...

Chunk 2

Refunds take...

Chunk 3

International shipping...

Each chunk gets converted into a numerical vector (an embedding).

You store:

Chunk text

Embedding vector

Document ID

Organization ID

Why chunking?

Because the LLM doesn't need an entire 50-page PDF.

If someone asks:

"How do refunds work?"

You only retrieve the refund-related chunks.

Customer Chat

A customer types:

Can I return shoes?

The flow is:

```

Question
↓
Embedding
↓
Vector Search
↓
Top 5 chunks
↓
Prompt Builder
↓
LLM
↓
Answer

```

Prompt Builder

This is one of the most important components.

Example prompt:

```

You are a customer support agent.
Use ONLY the provided context.
If the answer is unknown,
say you don't know.
Context:
Returns are allowed
within 30 days...
Refunds require receipt...
Question:
Can I return shoes?

```

Then send that prompt to Nemotron.

Conversation Memory

The model doesn't remember previous messages unless you provide them.

A common approach is:

```
Last 10 messages
+
Retrieved documents
+
Current question
```

That becomes the prompt.

Human Handoff

If the AI isn't confident:

```
"I don't know."
↓
Create support ticket
↓
Notify agent
↓
Human joins
```

Feedback

Every answer can be:

👍 Helpful

👎 Not Helpful

This helps identify gaps in the knowledge base.

Suggested Database Tables

```
organizations
users
documents
document_chunks
conversations
messages
feedback
```

support_tickets

Where Does Nemotron Fit?

Think of Nemotron as the reasoning engine. It doesn't store your company's knowledge.

It simply receives prompts like:

```
Context
+
Conversation
+
Question
```

and generates the best possible answer from that information.

Can You Skip RAG?

Yes, but only for very small knowledge bases.

For example:

```
FAQ
10 questions
↓
Hardcode into prompt
```

This works for tiny datasets.

Once you have:

- 100 FAQs
- 500 documents
- Thousands of pages

RAG becomes essential because you can't fit everything into the model's context window efficiently.

MVP Components

For a portfolio-worthy first version, I'd build these in order:

1. **Authentication & Organizations** (multi-tenant support)
2. **Knowledge Base** (upload PDFs, Markdown, and FAQs)
3. **Document Processing** (extract text, chunk, embed, and index)

4. **Customer Chat Widget** (embeddable on any website)
5. **RAG Pipeline** (retrieve relevant chunks and build prompts)
6. **LLM Integration** (Nemotron 3 Super via OpenRouter)
7. **Conversation History** (store chats and include recent messages)
8. **Admin Dashboard** (documents, chats, and basic analytics)
9. **Feedback & Human Handoff** (thumbs up/down and ticket creation)

This sequence lets you deliver a usable product quickly while adding progressively more advanced AI and SaaS features.

Can you explain more about what is RAG, how is that achieved?
Is it simply retrieval from docs
Do I need some tutorials or books before i move on?

This is one of the most important concepts in modern AI. The good news is that **RAG is much simpler than it initially sounds**. Many tutorials make it seem like magic, but it's essentially a smart search system combined with an LLM.

What is RAG?

RAG = Retrieval-Augmented Generation

Break it into two parts:

- **Retrieval** → Find the relevant information.
- **Generation** → Let the LLM generate an answer using that information.

The LLM itself is not retrained or fine-tuned.

Without RAG

Suppose you're using Nemotron.

You ask:

"What is Sam Suitings' return policy?"

The model has never seen your company's documents.

It might respond:

"Most companies offer returns within 30 days."

That's just a guess based on its general training.

With RAG

Now you've uploaded your company's handbook.

Return Policy

Customers may return products within 15 days.

Sale items cannot be returned.

Refunds are processed in 7 days.

When the user asks:

"Can I return a sale item?"

Your application first searches the uploaded documents and finds:

Sale items cannot be returned.

Then it builds a prompt like:

System:

You are a support agent.

Context:

Sale items cannot be returned.

User:

Can I return a sale item?

The model answers:

"No. According to the provided policy, sale items cannot be returned."

Notice that the model didn't "learn" your policy. It simply **used the retrieved text**.

So is RAG simply retrieving documents?

Yes—but with an important improvement.

It's not retrieving **whole documents**.

It's retrieving the **most relevant pieces** (called *chunks*).

For example, imagine a 200-page employee handbook.

The answer to "How many vacation days do I get?" might only require one paragraph.

Instead of sending all 200 pages to the LLM, you send only that paragraph.

That's what makes RAG practical.

How does retrieval work?

Imagine three documents:

```
Returns.pdf
Shipping.pdf
Warranty.pdf
```

Each is split into smaller chunks:

```
Returns
Chunk 1
Chunk 2
Chunk 3

Shipping
Chunk 1
Chunk 2

Warranty
Chunk 1
Chunk 2
```

When someone asks:

"How long does shipping take?"

You don't want all the chunks.

You only want the shipping-related ones.

The challenge is: **How do you find those automatically?**

The old way: Keyword search

Traditional search looks for matching words.

```
Shipping
↓
Find documents containing
"shipping"
```

This works until someone asks:

"When will my package arrive?"

The document might never mention the word "package"—it only says "shipping." A simple keyword search could miss it.

Embeddings solve this

Instead of storing words, you store their **meaning**.

An embedding model converts text into a list of numbers (a vector).

For example:

```
"Shipping takes 5 days."
```

↓

```
[0.18, -0.42, 0.77, ...]
```

The user's question is converted too:

```
"When will my package arrive?"
```

↓

```
[0.16, -0.40, 0.74, ...]
```

Even though the wording is different, the vectors are close because the meanings are similar.

This lets you find relevant information even without exact keyword matches.

Complete RAG pipeline

```
Upload PDF
```

↓

```
Extract text
```

↓

```
Split into chunks
```

↓

```
Generate embeddings
```

↓

```
Store chunks + embeddings
```

```
User asks question
```

↓

```
Generate embedding
```

↓

```
Vector search
```

↓

```
Top 5 chunks
```

↓

```
Build prompt
```

↓

```
Send to LLM
```

↓

Answer

The LLM only sees the retrieved chunks and the conversation—not the entire document collection.

What is a vector database?

A vector database stores:

- the chunk text
- its embedding (the vector)
- metadata (document ID, tenant ID, page number, etc.)

Then it can efficiently answer:

"Find the 5 chunks most similar to this question."

Many databases now support vector search directly, so you don't necessarily need a separate vector database for an MVP.

Is RAG difficult?

Not really. It has four core steps:

1. Read documents.
2. Split them into chunks.
3. Generate embeddings and store them.
4. Retrieve the best chunks before calling the LLM.

Most of the complexity is in engineering the pipeline well, not in the concept itself.

Do you need books first?

I don't think so.

RAG is best learned by building.

I would recommend this progression:

1. Understand prompts and how LLMs consume context.
2. Learn what embeddings are conceptually (you don't need the math).
3. Build a tiny RAG example with a few text files.
4. Add PDFs and chunking.
5. Add a vector search layer.
6. Improve retrieval quality and prompts.

That sequence teaches you much faster than reading an entire book.

Resources I'd recommend

Instead of a book, I'd focus on a few high-quality resources:

- **OpenAI Cookbook:** Excellent explanations of embeddings, vector search, and RAG patterns. Even if you use Nemotron through OpenRouter, the concepts are model-agnostic.
- **LangChain documentation:** Don't worry about learning the framework itself at first—its RAG tutorials clearly explain the workflow.
- **LlamaIndex documentation:** One of the best resources for understanding document ingestion, chunking, retrieval, and indexing, regardless of whether you use the library.
- **Sentence Transformers documentation:** Great for understanding embeddings and semantic similarity.

The key idea is to learn the concepts rather than becoming dependent on any one framework.

My recommendation

Given your experience with Next.js, Cloudflare Workers, databases, and APIs, I'd skip heavy theory and build a small RAG system yourself.

Start with just three Markdown files:

```
returns.md
shipping.md
faq.md
```

Then implement this flow:

```
Upload files
  ↓
Split into chunks
  ↓
Create embeddings
  ↓
Store them
  ↓
User asks a question
  ↓
Retrieve the top 5 chunks
  ↓
Send those chunks to Nemotron
  ↓
Display the answer
```

Once that works, you'll understand the core of RAG. From there, adding PDFs, websites, conversation memory, multi-tenancy, and analytics is mostly about extending the same pipeline rather than learning an entirely new concept.

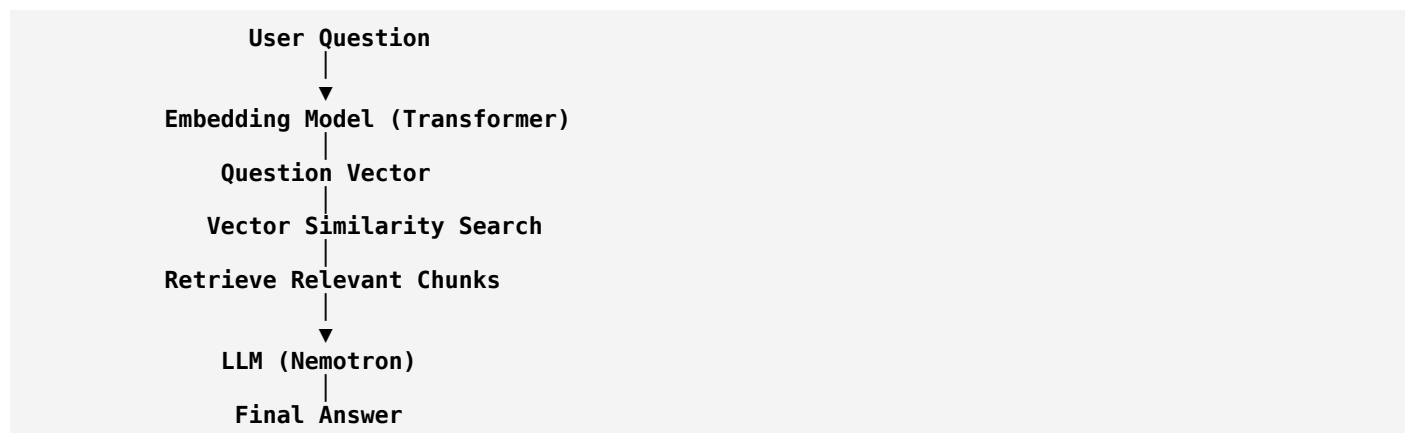
Generate embeddings

will some transformers do this part or LLM

Yes. This is where people often confuse **LLMs** with **embedding models**. They serve different purposes.

Task	Model Type	Example
Answer questions, write code, chat	LLM	Nemotron, Qwen, GPT-5, Llama
Convert text into vectors (embeddings)	Embedding model (Transformer)	BAAI/bge-large-en-v1.5, Nomic Embed, all-MiniLM-L6-v2

So in a RAG system, you typically have **two AI models**:



During document upload

Suppose you upload:

Returns.pdf
Customers may return items within 15 days.

Your application does:

```

Extract text
|
Split into chunks
|
Embedding Model
|
Produces vector:
[0.12, -0.45, 0.81, ...]
|
Store vector + chunk
  
```

Notice that **the LLM is not involved** here.

When a user asks a question

User:

"Can I return my shoes?"

Again, the embedding model runs:

```
"Can I return my shoes?"
↓
Embedding Model
↓
[0.10, -0.42, 0.80, ...]
```

Now your database compares that vector with all stored vectors and finds the closest matches.

For example:

```
Match 1 (99%)
Customers may return items within 15 days.
Match 2 (92%)
Refunds require original receipt.
```

Only **after retrieval** do you call the LLM:

```
System:
Use this context.

Context:
Customers may return items within 15 days.

User:
Can I return my shoes?
```

The LLM then writes a natural-language response.

Are embedding models Transformers?

Yes.

Most embedding models are built on the **Transformer architecture**, just like LLMs.

The difference is their objective:

- **LLMs** are trained to predict the next token (generate text).
- **Embedding models** are trained so that semantically similar texts produce nearby vectors in embedding space.

So they're related technologies, but optimized for different jobs.

Can an LLM generate embeddings?

Some can.

For example, some APIs expose embedding endpoints or hidden embedding representations from LLMs. However, for production RAG, it's generally better to use a dedicated embedding model because it is:

- Much faster
- Much cheaper
- More accurate for semantic search
- Designed specifically for retrieval

A useful mental model

Think of a RAG application as having three "brains":

1. **Embedding model** – *"Which documents are relevant?"*
2. **Vector database** – *"Find the closest matching chunks."*
3. **LLM** – *"Read those chunks and write a helpful answer."*

Each component has a distinct responsibility, and together they form the RAG pipeline.

AI Customer Support Agent

Before moving on solve bigger problem

lets start smaller as you have mentioned uploading .md files

and lets start headless

what do I need?

I actually think this is the **best way** to learn RAG. Forget PDFs, vector databases, SaaS, and dashboards for now.

Let's build the smallest possible RAG system.

No UI. No database. No frameworks. Just Node.js.

Goal

We want this interaction:

You: Can I return a damaged item?

↓

Your program

↓

Reads local markdown files

```
↓
Finds the relevant text
↓
Sends context to Nemotron
↓
Gets answer
```

That's it.

Step 0 - Project Structure

```
rag-demo/
knowledge/
  faq.md
  returns.md
  shipping.md
src/
  index.ts
  ingest.ts
  search.ts
  prompt.ts
  llm.ts
storage/
  chunks.json
package.json
```

Notice there is **no database**.

Step 1 - Write Knowledge

returns.md

```
Markdown
# Returns

Customers may return products within 15 days.

Sale items cannot be returned.

Refunds take 7 business days.
```

shipping.md

```
Markdown
# Shipping

Domestic shipping takes 3-5 days.

International shipping takes 7-14 days.
```

faq.md

Markdown

FAQ

Support hours:
Monday-Friday
9AM-5PM

These are your "company documents."

Step 2 - Read Files

Your first program should simply do:

```
knowledge/
↓
Read all .md files
↓
Print contents
```

No AI yet.

If you can do this, you've completed the first stage.

Step 3 - Chunk Documents

Instead of one huge string:

```
Returns
Customers may return products...
Sale items...
Refunds...
```

Split them into chunks.

Example:

```
JSON
[
  {
    "id":1,
    "text":"Customers may return products within 15 days."
  },
  {
    "id":2,
    "text":"Sale items cannot be returned."
  },
  {
    "id":3,
    "text":"Refunds take 7 business days."
  }
]
```



```
}  
]
```

Still no AI.

Step 4 - Generate Embeddings

Now the AI enters.

For every chunk:

Chunk

↓

Embedding Model

↓

Vector

Result:

JSON

```
{  
  "text": "Sale items cannot be returned.",  
  "embedding": [0.13, -0.82, ...]  
}
```

Store this inside

storage/chunks.json

Think of it as your tiny vector database.

Step 5 - Ask Question

User types

Can I return sale shoes?

Generate another embedding.

Question

↓

Embedding

↓

Vector

Step 6 - Similarity Search

Now compare the question vector with every stored vector.

Example:

```
Question
↓
Chunk 1
Similarity = 0.41

Chunk 2
Similarity = 0.98 ✓

Chunk 3
Similarity = 0.37
```

Winner:

```
Sale items cannot be returned.
```

This is the **Retrieval** part of RAG.

No LLM yet.

Step 7 - Build Prompt

Now you finally call Nemotron.

Prompt:

```
You are a helpful support agent.
Answer ONLY using this context.
Context:
Sale items cannot be returned.
Question:
Can I return sale shoes?
```

Nemotron replies

```
No.

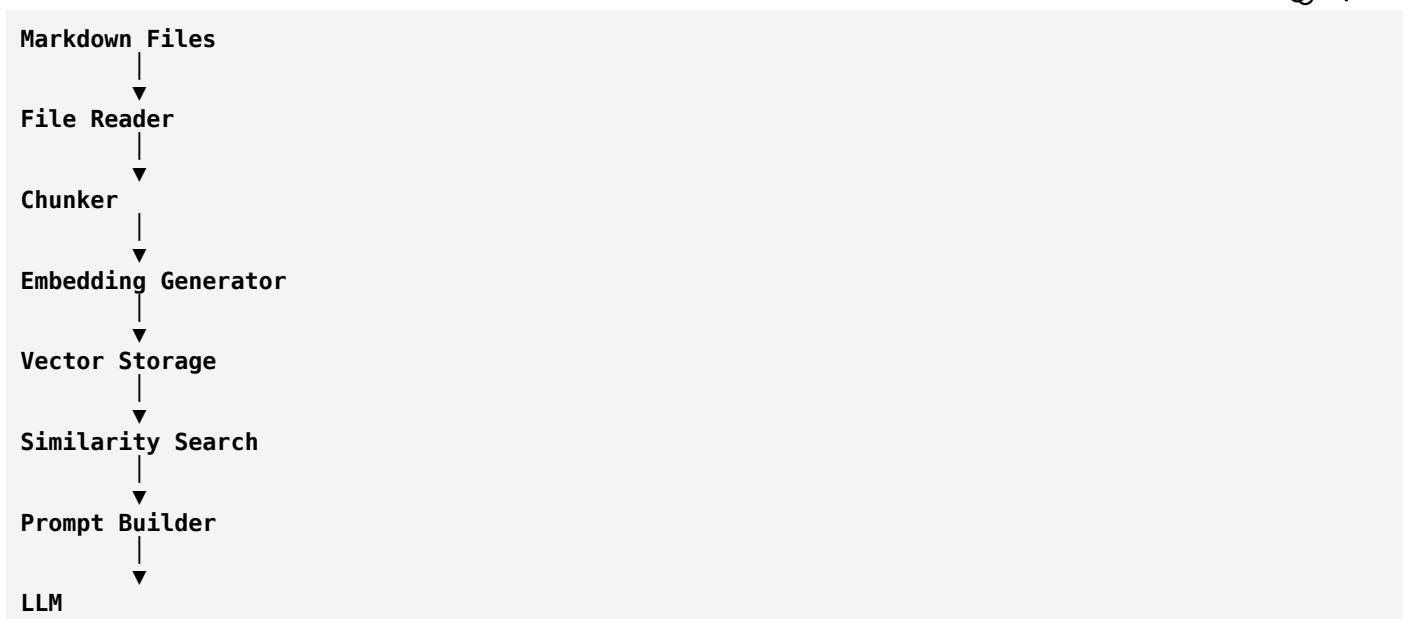
According to the return policy,
sale items cannot be returned.
```

Congratulations.

You have built RAG.

What Components Exist?

Notice how few there actually are.



Every enterprise RAG system—whether built by Microsoft, OpenAI, Anthropic, or startups—contains these same core stages. The surrounding infrastructure (authentication, scaling, monitoring, etc.) is what changes.

What libraries would we use?

For the first version, I'd keep dependencies to a minimum:

- **Node.js + TypeScript** — application
- **OpenRouter API** — Nemotron for text generation
- **An embedding model** (via an API or a local model) — to generate vectors
- **glob** — find .md files
- **gray-matter** (optional) — if you later want YAML front matter in Markdown
- **A cosine similarity function** — you can write this yourself in ~15 lines
- **JSON file** — store chunks and embeddings

Notice what's **not** on the list:

- **✗ LangChain**
- **✗ LlamaIndex**
- **✗ Pinecone**
- **✗ Chroma**
- **✗ PostgreSQL**
- **✗ Redis**

Those are excellent tools, but they're not necessary to understand RAG.

What we'll learn in this order

I would build this incrementally:

1. ✓ Read Markdown files.
2. ✓ Split them into chunks.
3. ✓ Generate embeddings.
4. ✓ Store them in `chunks.json`.
5. ✓ Implement cosine similarity yourself.
6. ✓ Retrieve the top 3 matching chunks.
7. ✓ Send those chunks to Nemotron.
8. ✓ Get an answer.

After completing those eight steps, you'll understand the essence of RAG. Everything else—PDF ingestion, vector databases, multi-tenancy, web search, hybrid retrieval, reranking—is an enhancement built on top of the same foundation.

I think this is a much stronger learning path than starting with LangChain, because you'll understand **why** each component exists before using frameworks that automate them.

LangChain

so LangChain do all this internally like from reading file to Retrieve the top matching chunks.

Yes, but with an important nuance.

LangChain is an orchestration framework, not an AI model.

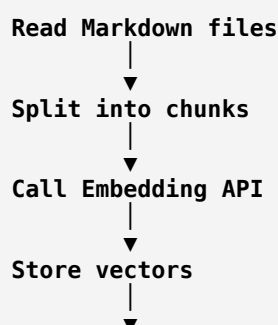
Think of it like **Express.js**.

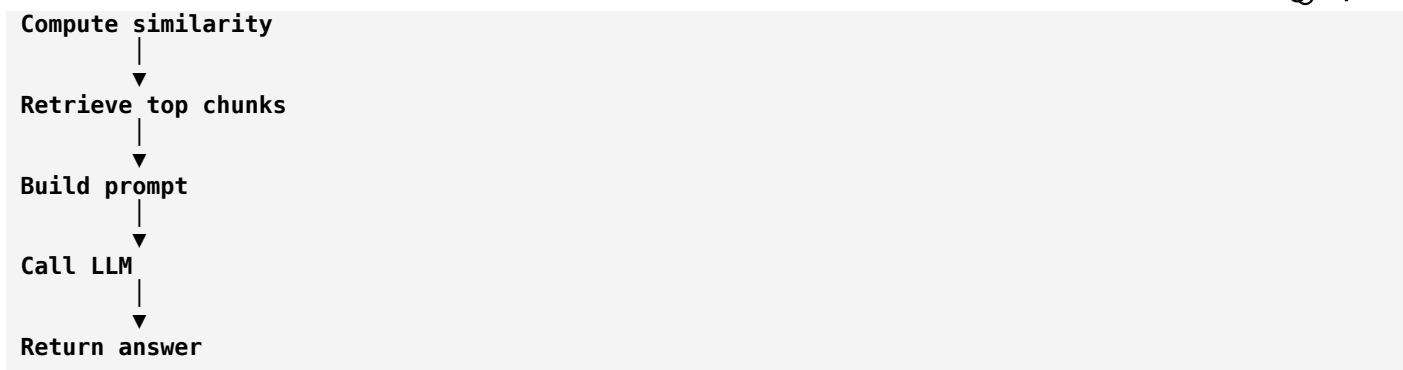
Express doesn't process payments, authenticate users, or talk to databases by itself. Instead, it connects those components together.

LangChain plays a similar role for LLM applications.

Without LangChain

You would write everything yourself:

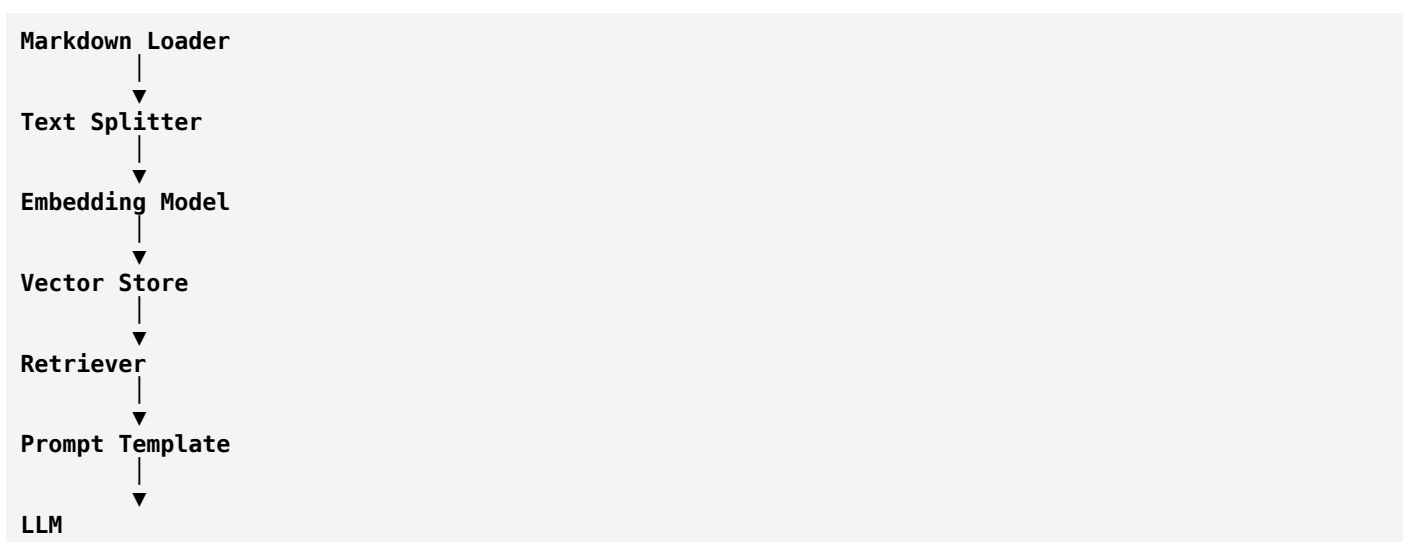




Every step is your own code.

With LangChain

LangChain provides implementations for many of these pieces.



You mostly configure components instead of writing them from scratch.

Mapping the concepts

Our Component	LangChain Component
Read .md files	MarkdownLoader
Read PDFs	PDFLoader
Chunk text	RecursiveCharacterTextSplitter
Generate embeddings	OpenAIEmbeddings, HuggingFaceEmbeddings, etc.
Store vectors	Chroma, Pinecone, FAISS, PGVector adapters
Similarity search	Retriever
Build prompts	PromptTemplate
Call Nemotron/OpenAI	LLM wrappers
Combine everything	Chains / Runnable

So yes, LangChain can cover almost the entire pipeline.

Example

Without LangChain

You might write:

```
TypeScript
const files = await readMarkdownFiles();
const chunks = splitIntoChunks(files);
const embeddings = await createEmbeddings(chunks);
await saveEmbeddings(embeddings);
const questionEmbedding = await createEmbedding(question);
const matches = cosineSimilarity(questionEmbedding, embeddings);
const prompt = buildPrompt(matches);
const answer = await callNemotron(prompt);
```

This teaches you exactly how RAG works.

With LangChain

The equivalent is much shorter:

```
TypeScript
const loader = new MarkdownLoader("./knowledge");
const docs = await loader.load();

const splitter = new RecursiveCharacterTextSplitter();
const chunks = await splitter.splitDocuments(docs);

const vectorStore = await MemoryVectorStore.fromDocuments(
  chunks,
  embeddings
);

const retriever = vectorStore.asRetriever();
const docs = await retriever.invoke(question);

const answer = await chain.invoke({
  question
});
```

Much less code, but most of the mechanics are hidden.

Does LangChain use AI internally?

Only where AI is actually needed.

For example:

Step	Uses AI?
Read file	✗ No
Parse Markdown	✗ No
Chunk text	✗ No
Generate embeddings	✓ Yes
Similarity search	✗ No (it's math on vectors)
Prompt template	✗ No
LLM answer	✓ Yes

This surprises many people.

Only **two stages** in a basic RAG pipeline require AI:

1. **Embedding model** (to convert text into vectors)
2. **LLM** (to generate the answer)

Everything else is ordinary software engineering.

Should you use LangChain immediately?

Given your background as a full-stack developer, I would **avoid LangChain for the first project**.

Here's why:

If you build the pipeline yourself once, you'll understand:

- what chunking actually does,
- why embeddings exist,
- how cosine similarity works,
- how retrieval happens,
- and what the prompt sent to the LLM looks like.

Once you understand those concepts, LangChain becomes a productivity tool rather than a black box.

My recommendation would be:

- **Project 1 (Learning):** Build the entire pipeline yourself with plain TypeScript and a JSON file as your vector store.
- **Project 2 (Production):** Introduce LangChain (or another orchestration library) where it genuinely saves time, such as supporting multiple document types, swapping embedding providers, or composing more complex workflows.

That way, you'll be comfortable both debugging a production RAG system and explaining exactly how it works in an interview or to a client.